

Reviewer Pack — Minimal Rank of Brauer Groups over Randomized Query Complexi...

Entry 47d6ed20428b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 02:59:32 UTC

Minimal Rank of Brauer Groups over Randomized Query Complexity of XOR-AND Trees

Entry ID: 47d6ed20428b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 02:59:32 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Brauer Groups) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

{‘text’: ‘For every XOR-AND tree T with n leaves, the minimal rank of the Brauer group of the ring of polynomials over the Boolean algebra $\{0,1\}$ modulo its ideal generated by the characteristic polynomial of T is at most $2^{(n/2)}$.’, ‘quantitative_relation’: ‘’, ‘falsifier’: ‘There exists an XOR-AND tree T with n leaves such that the minimal rank of the Brauer group of the ring of polynomials over $\{0,1\}$ modulo its ideal generated by the characteristic polynomial of T is greater than $2^{(n/2)}$.’}

Rationale (proposer’s reasoning):

{‘text’: ‘The connection between Brauer groups and Boolean rings has been studied in algebraic geometry and representation theory. The randomized query complexity of XOR-AND trees has applications in communication complexity, which could provide a bridge to analyze the structure of the polynomial ring modulo its characteristic ideal.’, ‘explanation’: ‘This conjecture attempts to establish a connection between abstract algebraic structures (Brauer groups) and computational complexity (query complexity). By exploring this relationship, one might uncover new insights into both fields.’}

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f5753de9d7187b77

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each XOR-AND tree T with n leaves ($5 \leq n \leq 40$), if the Spearman rank correlation coefficient between the minimal rank of the Brauer group and $2^{(n/2)}$ is greater than or equal to a predefined threshold, the conjecture is supported. If any seed produces a

correlation coefficient less than the threshold or a minimal rank greater than $2^{(n/2)}$, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Brauer Groups' AND 'XOR-AND tree width' AND minimality' - 'Representation Theory' AND XOR-AND trees AND 'Brauer group rank' - 'Complexity theory' AND 'randomized query complexity' AND 'Boolean algebra Brauer group'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def xor_and_tree(n):
    if n == 1:
        return [0, 1]
    left = xor_and_tree(n // 2)
    right = xor_and_tree(n - n // 2)
    return [x ^ y for x in left] + [x & y for x in left] + [x ^ y for x in right] + [x & y for x in right]

def characteristic_polynomial(tree):
    if len(tree) == 1:
        return tree[0]
    else:
        left_poly = characteristic_polynomial(tree[:len(tree)//2])
        right_poly = characteristic_polynomial(tree[len(tree)//2:])
        n = len(left_poly)
        result = [0] * (n + len(right_poly))
        for i in range(n):
            for j in range(len(right_poly)):
                result[i + j] += left_poly[i] * right_poly[j]
        return result

def minimal_rank_brauer_group(poly):
    n = len(poly)
```

```

if n == 1:
    return 0
matrix = [[0] * n for _ in range(n)]
for i in range(n):
    for j in range(n):
        if i != j:
            matrix[i][j] = poly[i] - poly[j]
rank = 0
for row in matrix:
    if any(row):
        rank += 1
        for other_row in matrix:
            if other_row != row and any(other_row):
                factor = other_row[rank-1] / row[rank-1]
                for k in range(n):
                    other_row[k] -= factor * row[k]
return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        tree = xor_and_tree(n)
        poly = characteristic_polynomial(tree)
        rank = minimal_rank_brauer_group(poly)
        results.append((n, rank))

    n_values = [r[0] for r in results]
    ranks = [r[1] for r in results]
    expected_ranks = [2**(n/2) for n in n_values]

    from scipy.stats import spearmanr
    correlation, _ = spearmanr(ranks, expected_ranks)

    metric_name = "Spearman rank correlation"
    metric_value = correlation
    instances_tested = len(n_values)
    conjecture_holds = correlation >= 0.95
    counterexample = "" if conjecture_holds else f"Correlation {correlation} < 0.95"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_

```

```

results.append(result)

mean_metric = sum(r['metric_value'] for r in results) / len(results)
std_metric = math.sqrt(sum((r['metric_value'] - mean_metric)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

if all(r['conjecture_holds'] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
    print(f"RESULT: FALSIFIED counterexample=\"Correlation < 0.95\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4836313.py", line 94, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4836313.py", line 62, in run_trial
    tree = xor_and_tree(n)
           ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4836313.py", line 21, in xor_and_tree
    left = xor_and_tree(n // 2)
           ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4836313.py", line 23, in xor_and_tree
    return [x ^ y for x in left] + [x & y for x in left] + [x ^ y for x in right] + [x & y for x in right]
           ^
NameError: name 'y' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support or falsification. | next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12757
2	preregistration	ollama_remote	glm4:latest	0	0	6203
3	novelty	ollama_remote	glm4:latest	0	0	5029
4	novelty	ollama_remote	glm4:latest	0	0	5606
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38875
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9321
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10900
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11856
9	judge	ollama_remote	glm4:latest	0	0	8063

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 108610 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/47d6ed20428b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/47d6ed20428b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/47d6ed20428b.tar.gz` (if generated)